

1. Image Analysis and Prediction

The app runs the optional classifier and independently reads color bands. When they disagree, a valid color-band value is displayed and the result is flagged for review.

src/App.tsx - classifier and color-band cross-check

```
const modelResult = await detectWithModel(image)

if (modelResult?.type === 'classification') {
  let bandCheck = null
  try {
    bandCheck = detectResistorBandsFromImage(image)
  } catch {
    bandCheck = null
  }

  const confidence = Math.round(modelResult.confidence * 100)
  const predictedOhms = parseResistanceLabel(modelResult.label)
  const disagrees = bandCheck && predictedOhms !== null &&
    Math.abs(predictedOhms - bandCheck.result.ohms) >
    Math.max(0.01, predictedOhms * 0.01)
  const bandEstimate = bandCheck?.result.formatted ?? ''

  setImageState({
    status: disagrees || !bandCheck ? 'warning' : 'success',
    bands: bandCheck?.bands ?? [],
    resistance: bandEstimate || modelResult.label,
    tolerance: bandCheck ? `+/-${bandCheck.result.tolerance}%` : '',
    bandEstimate,
    confidence,
    imageUrl: dataUrl,
  })
  return
}
```

2. Color-Band Detection and Value Calculation

The vision helper samples vertical colors inside the resistor body, groups stripe runs, and reads the sequence in either direction. The decoder rejects gold and silver in digit positions.

src/bandVision.ts - RGB hue classification

```
const saturation = maximum === 0 ? 0 : (maximum - minimum) / maximum
if (saturation < 0.42) {
  if (value < 0.18) return 'black'
  if (saturation < 0.1 && value > 0.88) return 'white'
  if (saturation < 0.16 && value > 0.55) return 'silver'
  return null
}

const delta = (maximum - minimum) / 255
let hue = 0
if (maximum === red) {
  hue = 60 * (((greenNorm - blueNorm) / delta) % 6)
} else if (maximum === green) {
  hue = 60 * ((blueNorm - redNorm) / delta + 2)
} else {
  hue = 60 * ((redNorm - greenNorm) / delta + 4)
}
hue = (hue + 360) % 360

if (hue < 10 || hue >= 350) return 'red'
if (hue < 35) return value < 0.62 ? 'brown' : 'orange'
if (hue < 49) return 'gold'
if (hue < 75) return 'yellow'
if (hue < 165) return 'green'
if (hue < 235) return 'blue'
return 'violet'
```

src/resistorCalculator.ts - decode four and five bands

```
export function decodeResistor(bands: string[], type: BandType) {
  const normalized = bands.map((band) => band.toLowerCase() as ResistorColor)
  const expectedCount = type === '4-band' ? 4 : 5
  if (normalized.length !== expectedCount ||
    normalized.some((color) => !(color in colorValues))) {
    throw new Error(`A ${type} resistor requires ${expectedCount} valid color bands.`)
  }

  if (type === '4-band') {
    const [a, b, multiplier, toleranceBand] = normalized
    if (!digitColors.has(a) || !digitColors.has(b)) {
      throw new Error('Gold and silver cannot be significant-digit bands.')
    }
    const ohms = (colorValues[a] * 10 + colorValues[b]) * colorMultiplier[multiplier]
    const tolerance = toleranceBand === 'gold' ? 5 : toleranceBand === 'silver' ? 10 : 20
    return { ohms, tolerance, formatted: formatResistance(ohms, tolerance) }
  }

  const [a, b, c, multiplier, toleranceBand] = normalized
  if (!digitColors.has(a) || !digitColors.has(b) || !digitColors.has(c)) {
    throw new Error('Gold and silver cannot be significant-digit bands.')
  }
  const ohms = (colorValues[a] * 100 + colorValues[b] * 10 + colorValues[c]) *
    colorMultiplier[multiplier]
  const tolerance = toleranceBand === 'gold' ? 5 : toleranceBand === 'silver' ? 10 : 20
  return { ohms, tolerance, formatted: formatResistance(ohms, tolerance) }
}
```

3. Training, Inference, and Test Results

The YOLO11 classifier is trained from the labeled resistor images. At runtime, Python returns the class label and confidence; pixel-based band decoding provides an independent value check.

scripts/train_resistor_classifier.py - model training and evaluation

```
model = YOLO('yolo11n-cls.pt')
model.train(
    data=str(data_dir),
    epochs=args.epochs,
    imgsz=args.imgsz,
    batch=args.batch,
    patience=10,
    seed=args.seed,
    project=str(PROJECT_ROOT / 'runs' / 'classify'),
    name='resistor-values',
    exist_ok=True,
)

best_weights = PROJECT_ROOT / 'runs' / 'classify' / 'resistor-values' / 'weights' / 'best.pt'
weights_path.parent.mkdir(parents=True, exist_ok=True)
shutil.copy2(best_weights, weights_path)
metrics = YOLO(str(weights_path)).val(data=str(data_dir), split='test')
print(f'Test top-1 accuracy: {metrics.top1:.4f}')
```

scripts/detect_resistor.py - runtime model prediction

```
model = YOLO(resolved_model)
image_size = 224 if model.task == 'classify' else 640
results = model(str(image_path), imgsz=image_size, verbose=False)

for result in results:
    if result.probs is not None:
        class_id = int(result.probs.top1)
        classification = {
            'label': str(result.names[class_id]),
            'confidence': float(result.probs.top1conf),
        }

print(json.dumps({
    'detections': detections,
    'classification': classification,
}))
```

src/bandVision.test.ts - the two resistor patterns

```
expect(detectColorBandSequence(makeResistorImage())).toEqual(
    ['yellow', 'violet', 'red', 'gold'],
)
expect(detectColorBandSequence(makeResistorImage([
    [210, 25, 24], [210, 25, 24], [124, 63, 15], [204, 160, 35],
]))).toEqual(['red', 'red', 'brown', 'gold'])
```

Classifier test result: 91% top-1 accuracy and 99.7% top-5 accuracy. The two sample color codes correspond to 4.7 kOhm and 220 ohms.